

Practica 9: Métodos Numéricos Para Encontrar Las Raíces De Una Función

Método de Newton-Raphson

Recordarán de la clase anterior que una forma de encontrar el 0 de una función es por medio del método de Newton-Raphson. Como vieron en la clase, este método está dado por la siguiente relación de recurrencia:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Pero, ¿Qué nos está diciendo el método de Newton-Raphson geométricamente?

La idea subyacente del método es linealizar la función en cada paso y tomar la recta que pasa por $(x_n, f(x_n))$ con pendiente $f'(x_n)$; es decir, tomar la recta tangente a la gráfica en x_n . Luego, nos movemos en la dirección determinada por dicha recta hasta intersectar el eje x y el punto determinado por dicha intersección será nuestro x_{n+1} . Para entender mejor la idea geométrica del método supongamos que nuestra aproximación inicial a la raíz de una función f es x_0 . Entonces tomamos la recta tangente a f en x_0 y nos movemos sobre ella hasta intersectar el eje de las x . A dicho punto de intersección lo llamamos x_1 y repetimos el proceso para encontrar x_2 . Así, sucesivamente, nos vamos moviendo sobre las rectas tangentes a la gráfica e intersectando el eje de las x para encontrar los nuevos puntos:

Ahora, como se imaginarán, no siempre tenemos una convergencia monótona a nuestra raíz. Por ejemplo, cuando aplicamos el método a la arcotangente con $x_0=1.39$ obtenemos lo siguiente:

Más aún, a veces ni siquiera tenemos convergencia. Por ejemplo, si aplicamos el método a la arcotangente pero ahora con $x_0=1.4$ obtenemos lo siguiente:

Moraleja: La decisión de nuestra aproximación inicial influye en el método, por lo que vale la pena intentar empezar lo más cerca de la raíz que queremos encontrar.

Ejercicio:

Calcularemos las dos raíces del polinomio $p(x)=2x^4+24x^3+61x^2-16x+1$ cerca de $x=0.1$ usando el método de Newton-Raphson y el método del punto fijo (o método de aproximaciones sucesivas) y calcularemos la eficiencia de los dos.

Método de Mewton-Raphson

Paso 1:

Programa una función que aplique el método de Newton-Raphson a una función en general. Tu función debe tener dos criterios de paro. El primero será la tolerancia de algoritmo, en la cual especificamos que tan pequeño queremos hacer el valor de f . Es decir, si la tolerancia es ε se deberá de cumplir que $|f(x_n)| < \varepsilon$ para que pare. El otro criterio de paro deberá ser un número máximo de iteraciones en caso de que nunca se alcance la tolerancia. Además debe recibir la condición inicial, x_0 , la función f y la función f' . Finalmente, debe regresar una lista con todos los valores de las iteraciones.

Paso 2:

Como se vio en la sección anterior, el resultado del método depende de la condición inicial. Por ello vale la pena echar un vistazo a la gráfica para saber con que condiciones iniciar el método. En este ejercicio deberán graficar $P(x)$ para ver aproximadamente en que valores se encuentran las raíces cercanas a $x=0.1$. Una vez que hayan determinado dichos valores prueben su función con las diferentes condiciones iniciales y evalúen $P(x)$ en los valores obtenidos para corroborar que sí son ceros.

Prueba tu función con una tolerancia de 10^{-10} y un número máximo de iteraciones de 10,000.

Pista : Deben obtener dos valores distintos en el intervalo $[0.12, 0.125]$.

```
import numpy as np
import matplotlib.pyplot as plt
```

Comienza aquí tu código

Método de aproximaciones sucesivas

La idea de este método recae en el siguiente teorema:

Teorema:

Sea $g: [a, b] \rightarrow [a, b]$ una función de clase C^1 en $[a, b]$ tal que $|g'(x)| \leq k < 1 \forall x \in [a, b]$. Entonces existe un único $x^* \in [a, b]$ tal que $g(x^*) = x^*$. Además, si $x_0 \in [a, b]$ y definimos $x_n = g(x_{n-1})$ entonces $x_n \rightarrow x^*$.

Así, la idea es buscar una función g cuya derivada sea siempre menor que 1 en un intervalo que contenga a cada una de las raíces y que además tenga como punto fijo a la raíz de P , es decir que $g(x^*) = x^* \Leftrightarrow P(x^*) = 0$. Así, en vista del teorema anterior, tenemos que la sucesión dada por

$$x_{n+1} = g(x_n)$$

convergerá a una de las raíces de P

Paso 1:

Encuentra unas funciones g_1 y g_2 que cumplan que $|g_1'(x)| \leq k < 1 \forall x \in [a, b]$ y $|g_2'(x)| \leq k < 1 \forall x \in [c, d]$ de forma que $[a, b]$ contenga a la primera raíz que encuentraste antes y que $[c, d]$ contenga a la segunda raíz que encuentraste antes. Además se debe cumplir que $g(x_1) = x_1 \Leftrightarrow P(x_1) = 0$ y $g(x_2) = x_2 \Leftrightarrow P(x_2) = 0$

Paso 2:

Programa una función que aplique el método del punto fijo con una función g en general. Tu función debe tener dos criterios de paro. El primero será la tolerancia de algoritmo, en la cual especificamos que tan pequeño queremos hacer el valor de f . Es decir, si la tolerancia es ε se deberá de cumplir que $|g(x_n) - x_n| < \varepsilon$ para que pare. El otro criterio de paro deberá ser un número máximo de iteraciones en caso de que nunca se alcance la tolerancia. Además debe recibir la condición inicial, x_0 , y la función g . Finalmente, debe regresar una lista con todos los valores de las iteraciones.

Prueba tu función con las g_1 y g_2 obtenidas anteriormente y con una tolerancia de 10^{-10} y un número máximo de iteraciones de 10,000.

Comparación

Imprime las longitudes de las listas de iteraciones del método de Newton-Raphson y del Punto Fijo para cada una de las raíces y compara la longitud. Esto nos dirá cuantas iteraciones le tomó a cada método alcanzar la tolerancia y si alcanzó la tolerancia o no. Con dicha información evalúa cuál de los métodos fue mejor e intenta dar una explicación a ese resultado.

Apéndice

Para jugar con el método de Newton-Raphson

En el siguiente link pueden probar el método con distintas funciones y diferentes números de iteraciones y ver como se comporta gráficamente.

<https://www.geogebra.org/m/XCrwWHzy>

Un poco de código en R para hacer animaciones

La animación del principio se hizo en R usando el siguiente código:

```
# Instalamos la paquetería
install.packages("animation")
# Cargamos la paquetería
library(animation)

# Aquí corremos la función
par(pch = 20)
ani.options(nmax = 50)
newton.method(function(x) x^2, init = 100, c(0, 105))

# Aquí guardamos el resultado como un gif
saveGIF(newton.method(function(x) x^2, init = 100, c(0, 105)),
movie.name = "x2.gif", img.name = "Rplot", convert = "magick",
cmd.fun, clean = TRUE)
```

para más información pueden ir a:

<https://www.rdocumentation.org/packages/animation/versions/2.6/topics/newton.method>

```
# Instalamos la paquetería
install.packages("animation")
# Cargamos la paquetería
library(animation)
```

```
# Aquí corremos la función
par(pch = 20)
ani.options(nmax = 50)
newton.method(function(x) x^2, init = 100, c(0, 105))
```

```
# Aquí guardamos el resultado como un gif
saveGIF(newton.method(function(x) x^2, init = 100, c(0, 105)),
movie.name = "x2.gif", img.name = "Rplot", convert = "magick",
cmd.fun, clean = TRUE)
```

package 'magick' successfully unpacked and MD5 sums checked

Warning message:

```
"cannot remove prior installation of package 'magick'"Warning message
in file.copy(savedcopy, lib, recursive = TRUE):
"problema al copiar C:\Users\brend\anaconda3\Lib\R\library\00LOCK\
magick\libs\x64\magick.dll a C:\Users\brend\anaconda3\Lib\R\library\
magick\libs\x64\magick.dll: Permission denied"Warning message:
"restored 'magick'"
```

The downloaded binary packages are in

C:\Users\brend\AppData\Local\Temp\RtmpsJIrkz\downloaded_packages

Warning message:

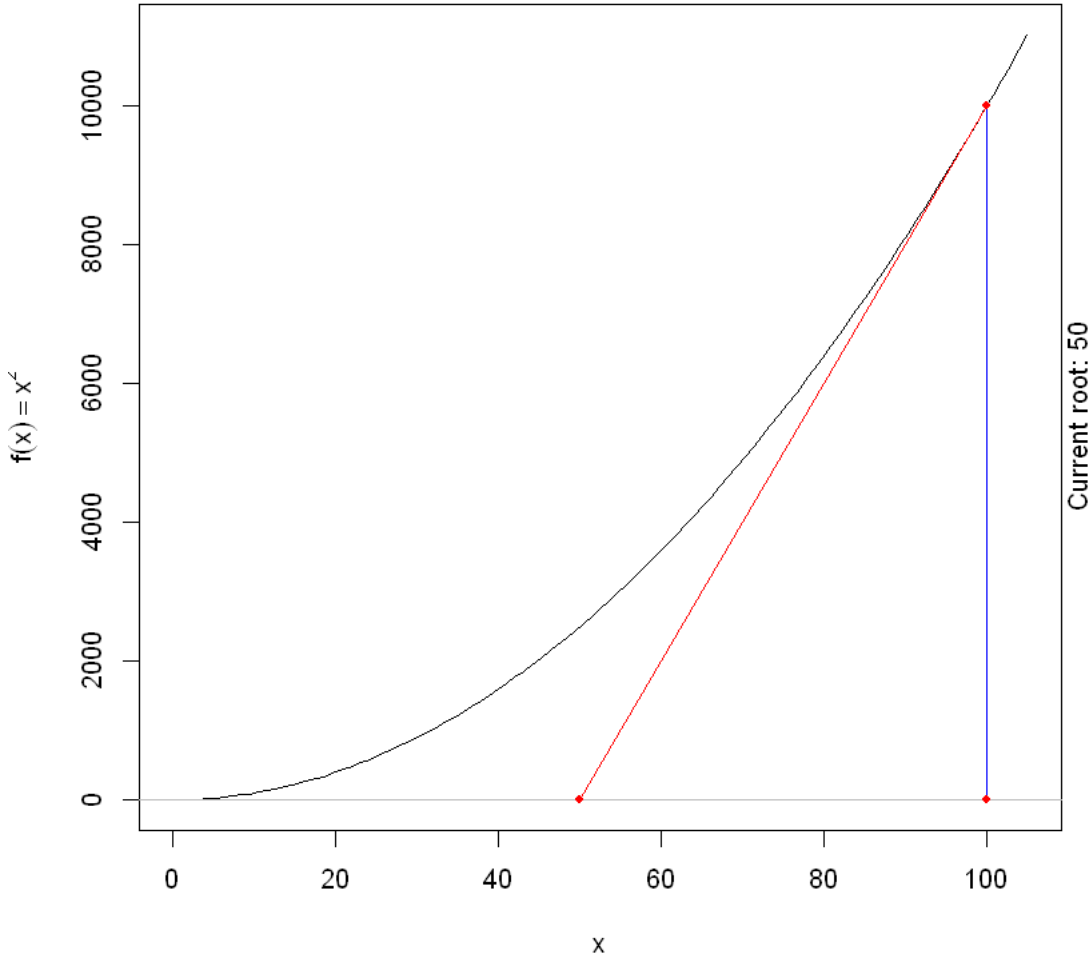
```
"package 'animation' is in use and will not be installed"Warning
message:
```

```
"package 'magick' was built under R version 3.6.3"Linking to
ImageMagick 6.9.11.34
```

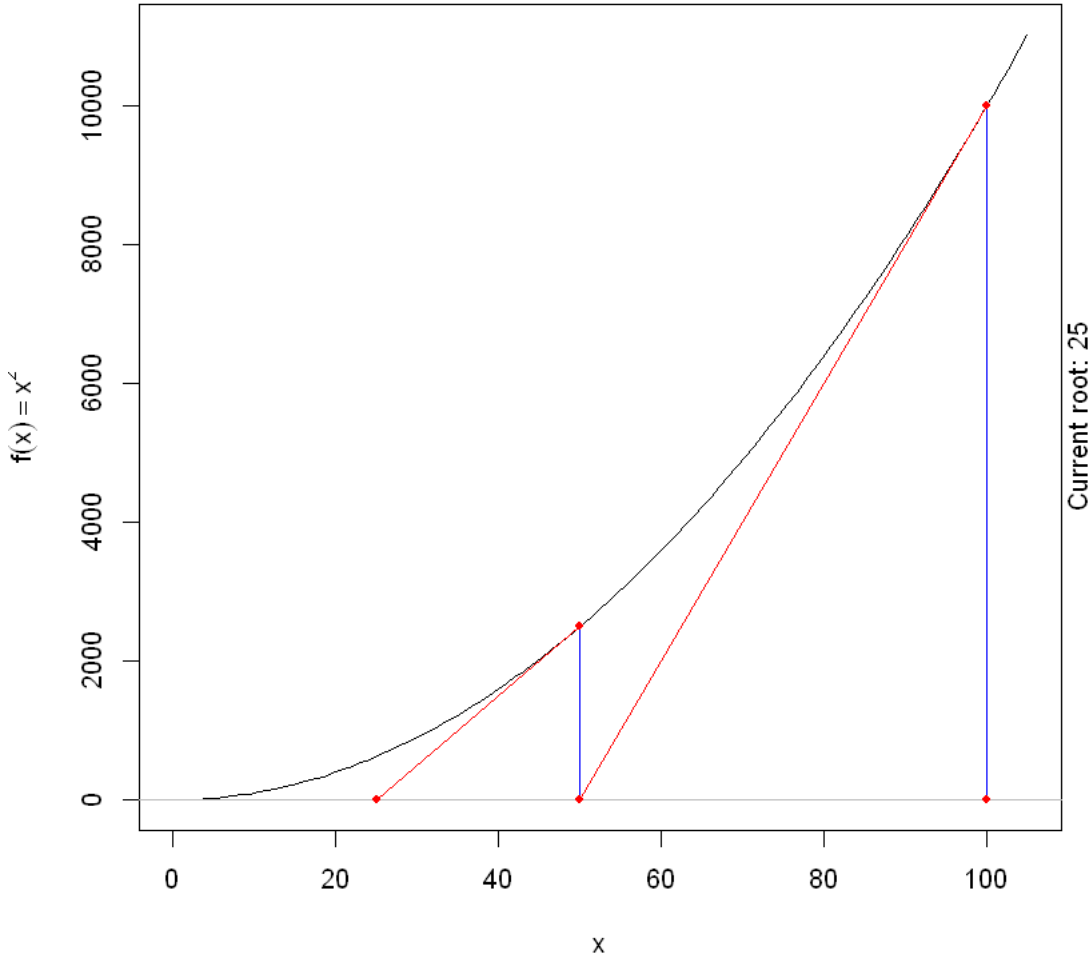
```
Enabled features: cairo, freetype, fftw, ghostscript, lcms, pango,
rsvg, webp
```

```
Disabled features: fontconfig, x11
```

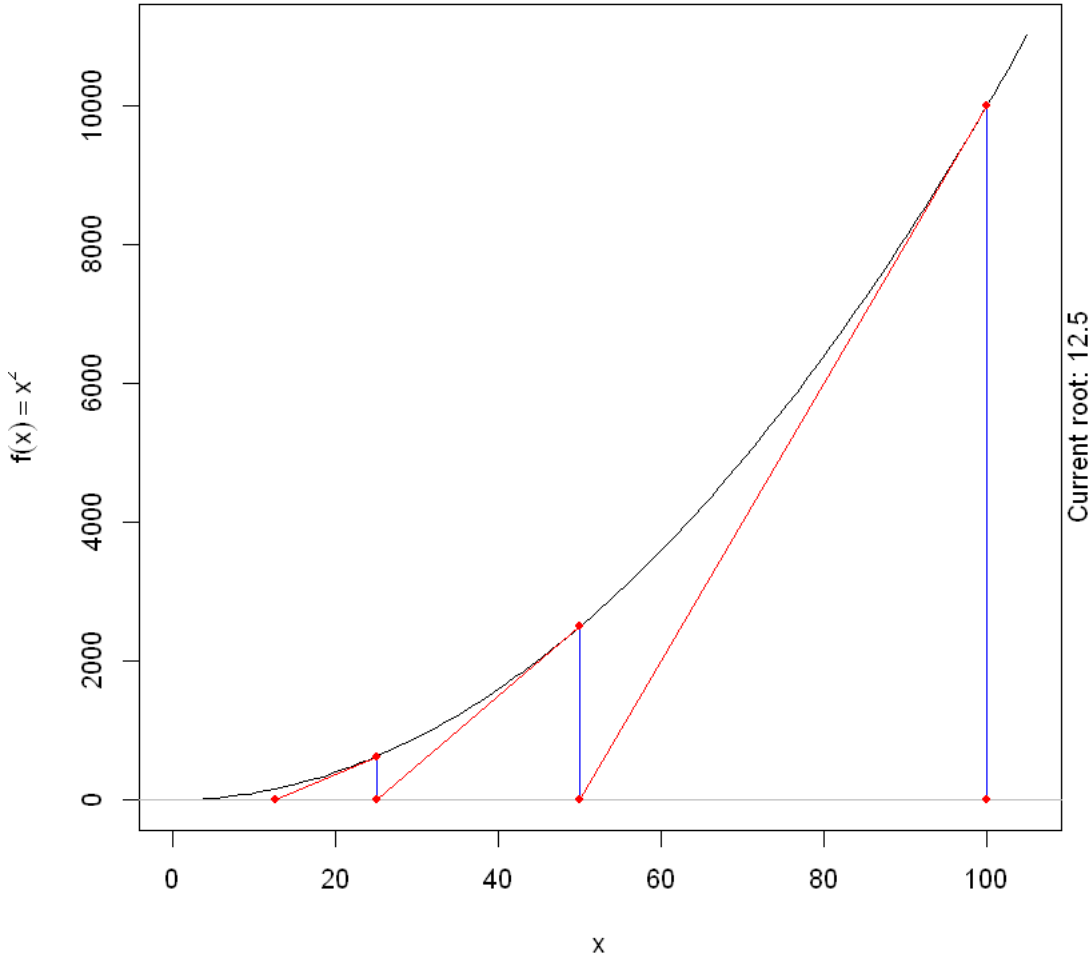
Root-finding by Newton-Raphson Method: $x^2 = 0$



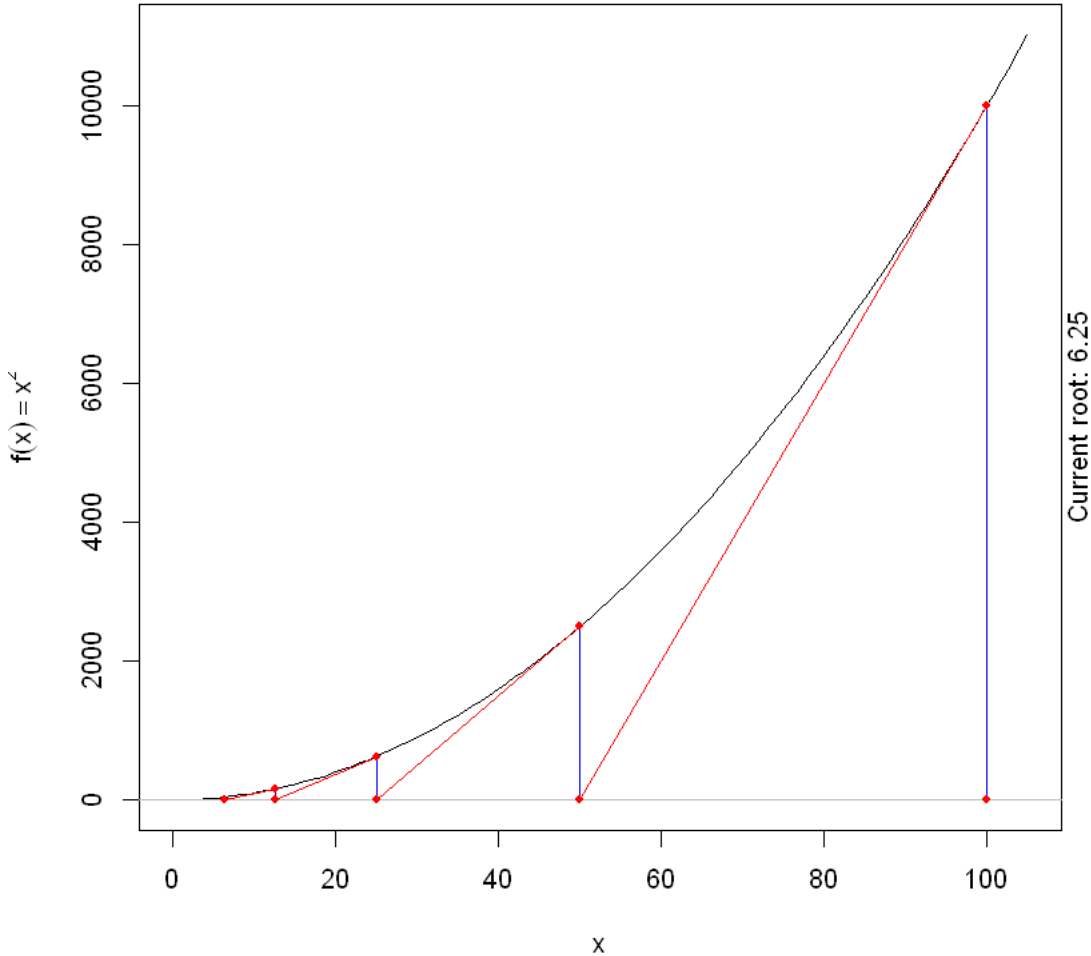
Root-finding by Newton-Raphson Method: $x^2 = 0$



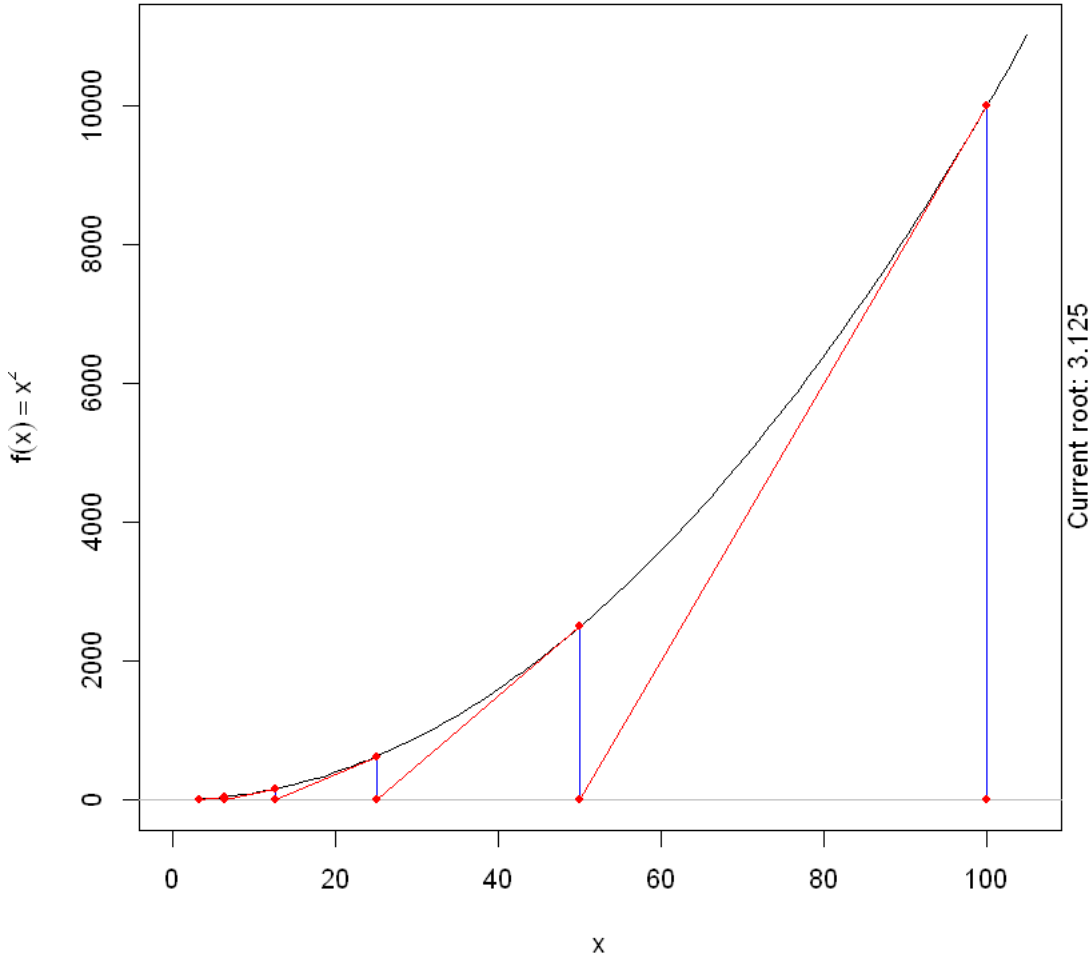
Root-finding by Newton-Raphson Method: $x^2 = 0$



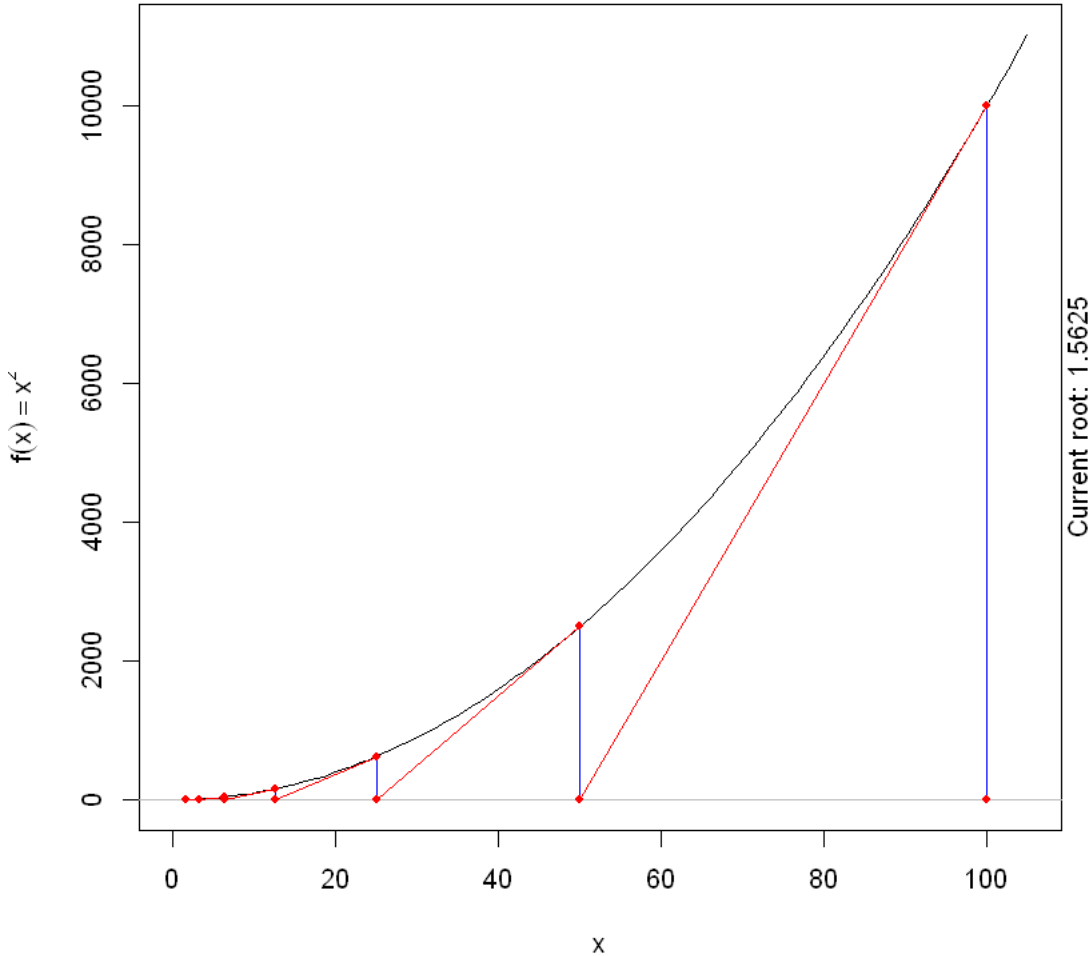
Root-finding by Newton-Raphson Method: $x^2 = 0$



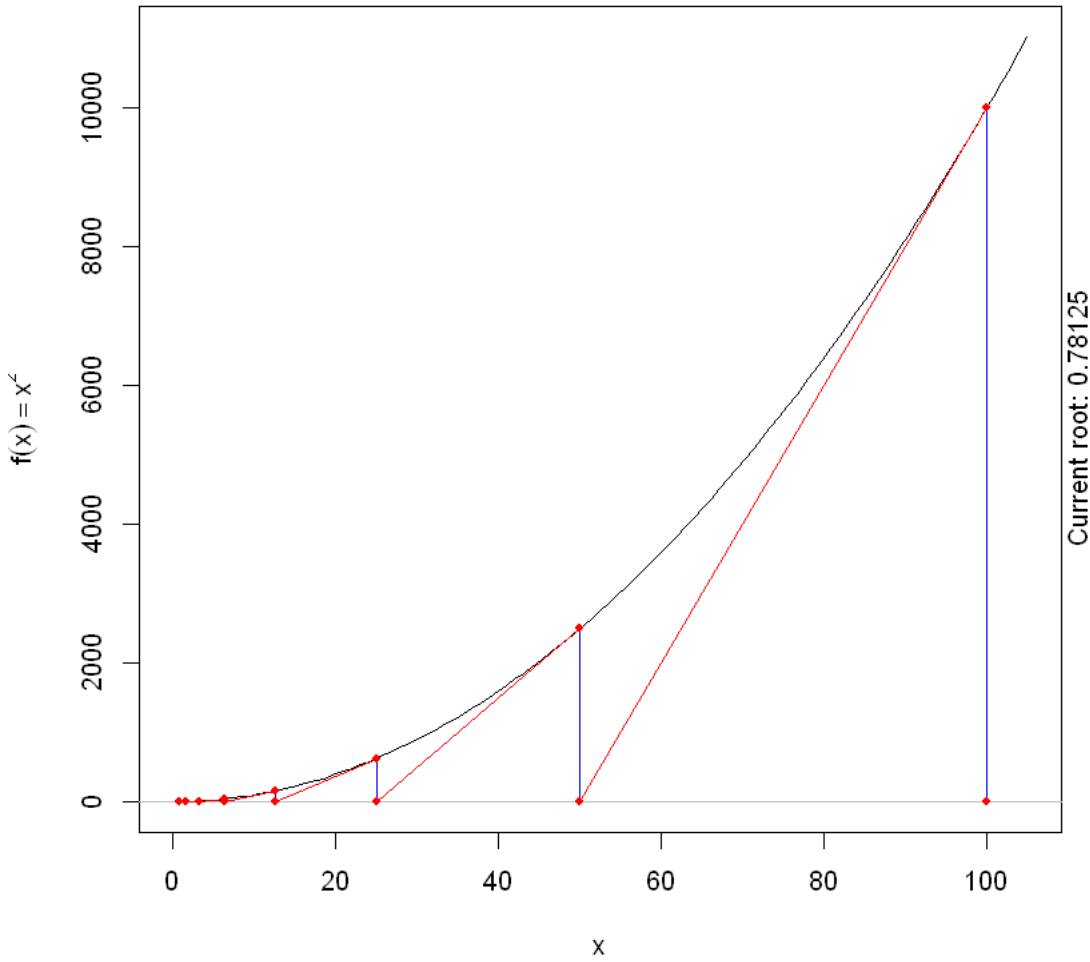
Root-finding by Newton-Raphson Method: $x^2 = 0$



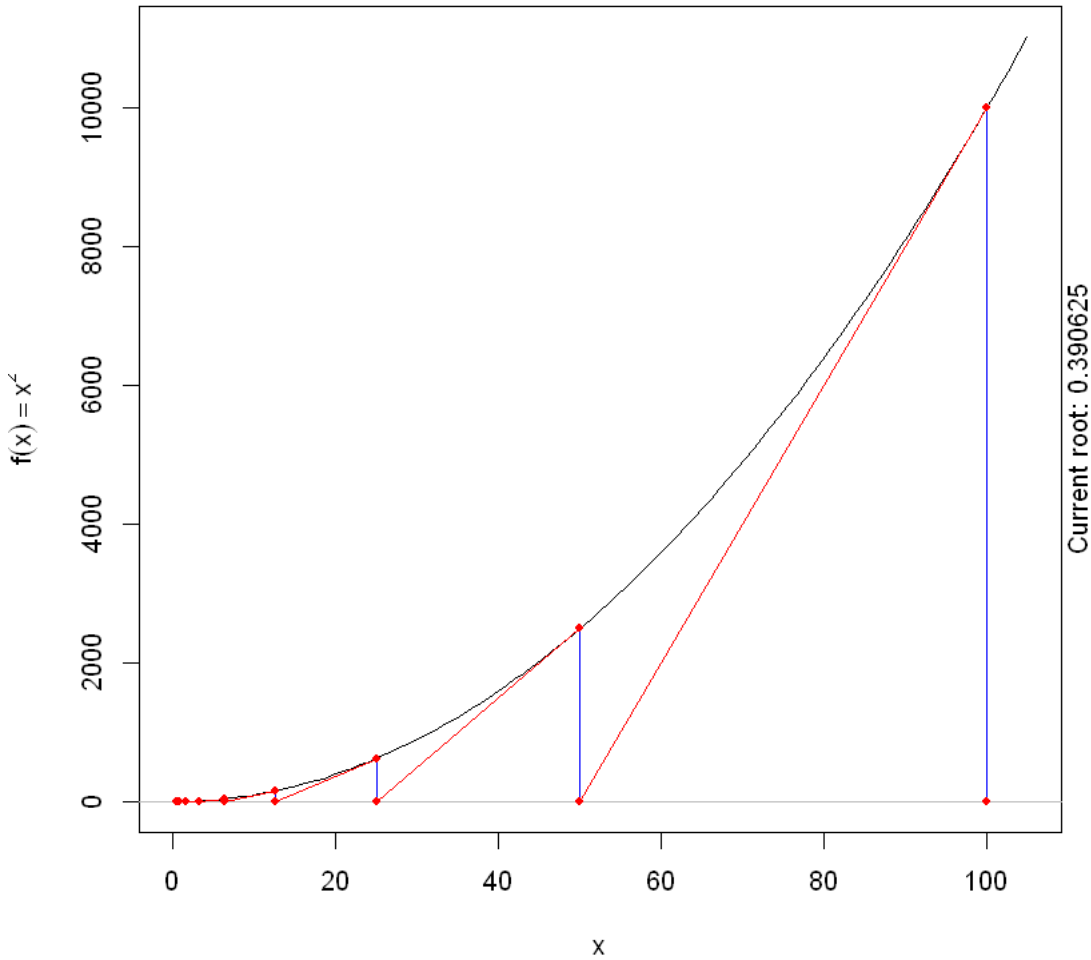
Root-finding by Newton-Raphson Method: $x^2 = 0$



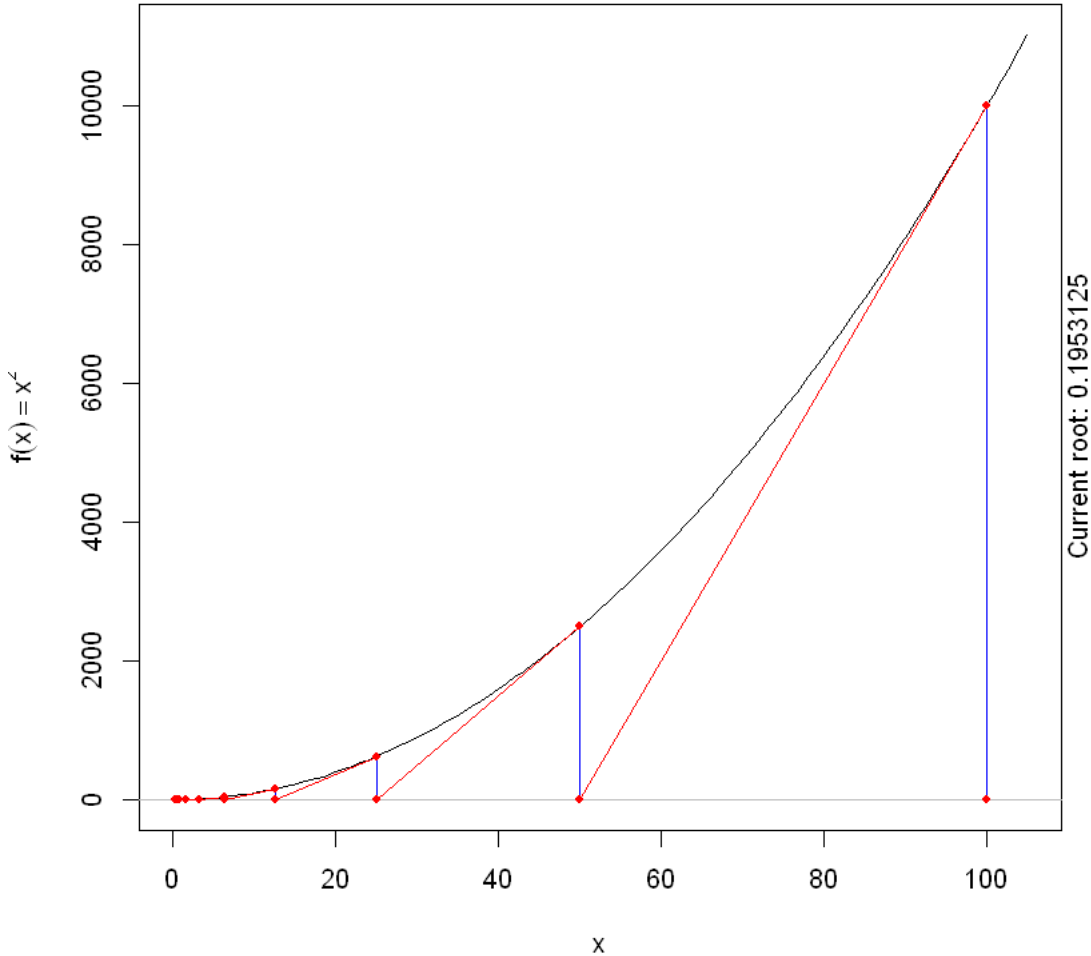
Root-finding by Newton-Raphson Method: $x^2 = 0$



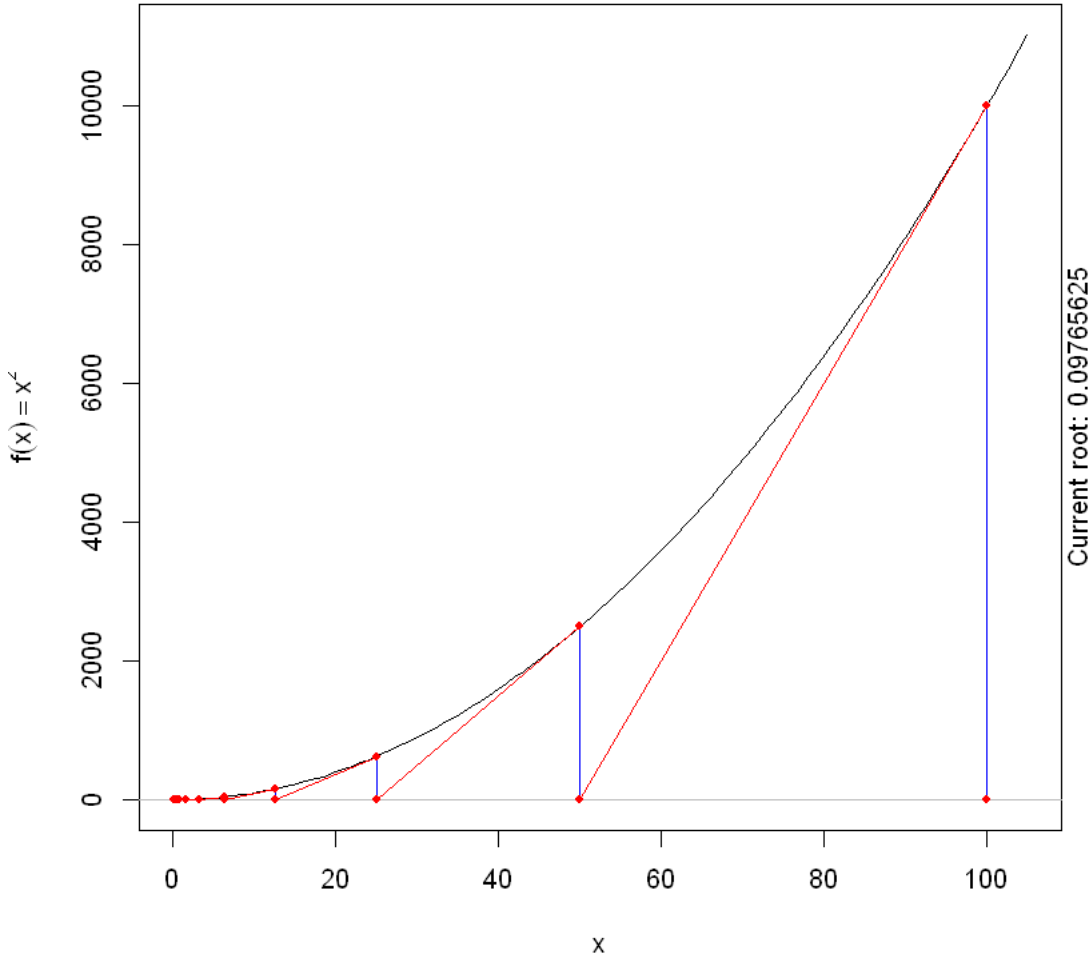
Root-finding by Newton-Raphson Method: $x^2 = 0$



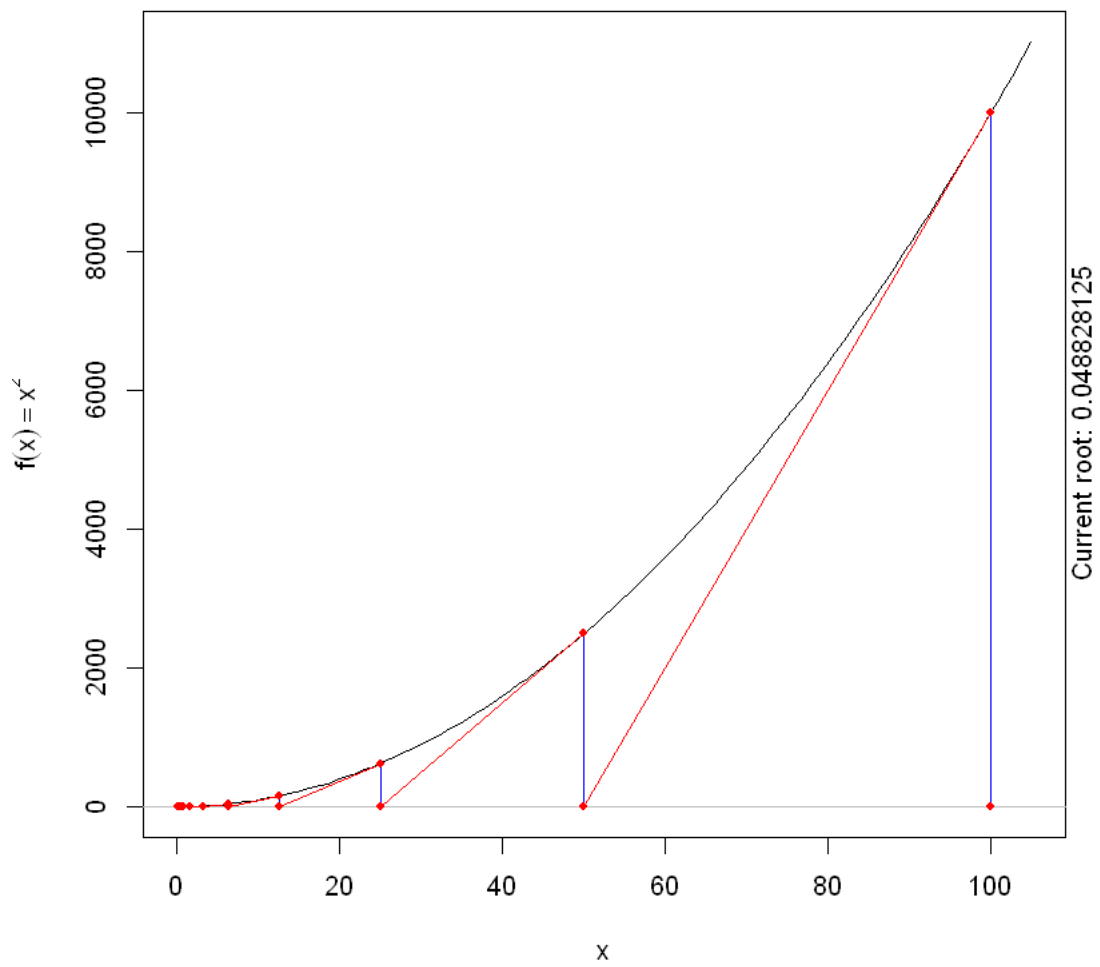
Root-finding by Newton-Raphson Method: $x^2 = 0$



Root-finding by Newton-Raphson Method: $x^2 = 0$



Root-finding by Newton-Raphson Method: $x^2 = 0$



Output at: x2.gif

[1] TRUE

Root-finding by Newton-Raphson Method: $x^2 = 0$

